

Software Testing Plan



March 29th, 2026

KIMinder360 Team

Dylan Hyer (deh277), Nyle Huntley (nah353),
Cole Bishop (cdb499), Mayanna John (mrj335)

Morgan W. Boatman (Sponsor)

Md Nazmul Hossain (Mentor)

Version 1.7

Table Of Contents

Introduction	3
Unit Testing	4
Integration Testing	8
Key Integration Points	9
Test Environment and Setup	9
Integration Scenarios	10
Usability Testing	12
Testing Workflow and Quality Controls	14
Bug Reporting Requirements	14
Severity Levels	14
Development Cycle Controls	15
Conclusion	16

Introduction

KIMinder360 is a mobile application designed to help users develop stronger situational awareness through short, real-world training challenges delivered throughout the day. The application automates the delivery of these challenges, known as directives, by sending notifications to users during customizable active hours. Users can configure directive categories, interaction levels, and notification frequency while the application tracks their responses and engagement locally on the device. The primary intended users of KIMinder360 are individuals interested in improving personal awareness and attentiveness in everyday environments. Because the application operates as a background-driven mobile system that relies on scheduled notifications and local data persistence, its most important quality goals are reliability, correctness of directive scheduling and delivery, usability of the mobile interface, and efficient performance on resource-constrained devices.

From a testing perspective, KIMinder360 is implemented as a native Android application following a layered Model-View-ViewModel (MVVM) architecture. The system consists of a user interface layer built with Jetpack Compose, an application logic layer responsible for directive selection and scheduling, a local data layer implemented with the Room database and DataStore, and a notification subsystem using Android's NotificationManager and WorkManager APIs. Future functionality may involve a cloud synchronization layer using Firebase. Testing will therefore focus primarily on the interactions between these internal components as well as their integration with Android system services responsible for background execution and notifications.

The testing environment includes Android Studio development tools, Android emulators, and physical Android devices running the supported Android versions. The scope of testing includes directive scheduling logic, user preference management, database persistence, notification delivery, and user interface behavior. Out of scope items include the optional cloud synchronization layer, iOS compatibility, and any external services that are not part of the current minimum viable product. Testing activities are aligned with the project's development phases and demonstration milestones, ensuring that core features implemented during each phase are validated before demonstrations.

The overall testing strategy for KIMinder360 involves unit testing, integration testing, and usability testing to validate both the internal correctness of system components and the overall user experience. Unit testing will be used to verify individual functions and modules, particularly those responsible for directive selection, scheduling logic, and preference management. Integration testing will evaluate interactions between components such as the ViewModels, database layer, and background scheduling system to confirm that directives are selected, scheduled, stored, and displayed

correctly across layers. Usability testing will focus on evaluating the clarity and accessibility of the interface, particularly for settings configuration, directive interactions, and navigation through the application. Testing will be performed incrementally throughout development, with unit tests being conducted after each new pull request, integration tests being conducted prior to all demonstrations, and usability testing planned before the Acceptance demo.

The chosen testing strategy reflects the risks and priorities inherent to the KIMinder360 system. Because directive delivery depends on accurate scheduling, background execution, and persistent storage, failures in these areas could significantly impact user trust and engagement. As a result, the application logic layer, scheduling mechanisms, and database interactions receive particular testing focus. User interface components also require validation to ensure that customization options and directive responses remain intuitive and accessible to users. Meanwhile, less complex or impactful features require comparatively less intensive testing at this stage of development. By focusing testing effort on the most critical and risk-sensitive parts of the system, the team can ensure the reliability and usability of the application while operating within the limited development timeline. The following sections will further describe the testing processes, test environments, and evaluation criteria that will be used to validate the KIMinder360 system.

Unit Testing

Unit testing focuses on verifying the correctness and reliability of the core business logic, particularly in areas involving directive selection, scheduling behavior, and user-generated data handling. The primary goals of these tests are to validate input handling, ensure proper data transformations, and confirm that critical computations like filtering and persisting user responses behave as expected under normal, boundary, and edge-case conditions. Unit tests will be implemented using standard Android Studio testing tools, including JUnit, MockK, and Kotlin Coroutines Test, and will be executed locally before each pull request is made.

Key testing metrics will include test pass/fail status and code coverage, focusing primarily on the most critical components. The main units under test include methods within the DirectiveRepository (database) and ViewModel (UI) classes, such as directive selection and scheduling, personal directive creation, and response submission. Testing efforts are focused on these areas due to their relative complexity and impact on user experience, while simple UI rendering code and data access objects are excluded since they involve minimal internal logic.

Unit Under Test: getRandomActiveDirective(context) (in DirectiveRepository)

Purpose: This is the core scheduling logic of the application. It manages the complex interaction between user-enabled categories and the "recent directives" filter. Failure here would result in the app failing to send reminders or repeatedly sending the same ones, breaking the primary user experience.

Test Case Categories:

- *Valid inputs:* Multiple active categories with varied directive counts.
- *Boundary cases:* Only one active category; all directives in a category have been seen recently (testing fallback).
- *Invalid/Edge cases:* No categories marked as active; no directives exist in the database at all.

Sample Tests:

- Returns a valid directive when active categories exist.
- Returns null properly when no categories are enabled.
- Successfully falls back to any random directive when the "non-recent" filter returns an empty list.

Unit Under Test: createNewPersonalDirective(text) (in PersonalDirectivesViewModel)

Purpose: This handles user-generated content. Without validation, the database could be corrupted with invalid entries, or the UI could break if a user enters extremely long text or incompatible characters.

Test Case Categories:

- *Valid inputs:* Standard alphanumeric strings; text with emojis/symbols.
- *Boundary cases:* Single character inputs; strings at the maximum expected length (e.g., 255-500 chars).
- *Invalid inputs:* Empty strings; strings containing only whitespace; extremely long bomb strings; SQL injection-style characters (e.g., ' OR 1=1; --).

Sample Tests:

- Creating a directive with standard text correctly links it to the "Personal Directives" category.
- Submitting a very long string (e.g., 5000+ characters) is truncated or rejected.
- Submitting a string with emojis or SQL statements is handled appropriately without any impact on the database.

Unit Under Test: submitResponse(isDone, response) (DirectiveResponseViewModel)

Purpose: This records the user's progress. Similar to personal directives, it is one of the few places where free-text input (the user's response) is saved to the database. We need to ensure that different response types (empty, long, or special characters) are handled without crashing the update process or corrupting the history log.

Test Case Categories:

- *Valid inputs:* Completing a directive with a text response; Skipping a directive with a null/empty response.
- *Boundary cases:* Response at maximum character limit; response containing only emojis or newlines.
- *Edge cases:* Submitting a response for a directive ID that has been deleted or is invalid.

Sample Tests:

- Submitting "Done" with a valid comment correctly updates the database and refreshes the UI state.
- Submitting a response with special characters (e.g., symbols or emojis) is stored correctly.
- Submitting a very long response is handled correctly by the DAO.

Unit Under Test: updateCompletion(id: Int, isDone: Boolean)

Purpose: This test is designed to make sure everything needed for the update completion function is done. This includes making sure the DAO method is called, making sure the coroutine launches, and making sure the correct measures are passed.

Test Case Categories:

- *Valid inputs:* Updating a directive's completion status from incomplete to complete; Updating a completion status from complete back to incomplete.
- *Boundary cases:* Updating completion status for the lowest valid directive ID; Updating completion status for the highest valid directive ID.
- *Edge cases:* Updating completion status for a directive ID that does not exist; Updating completion status when the database response text is null.

Sample Tests:

- Updating the completion status with a valid directive ID correctly calls the DAO update method.
- Updating the completion status from true to false correctly persists the new value.
- Updating the completion status for a directive with no stored response does not crash the update process.

Unit Under Test: recentDirectives: StateFlow<List<Pair<RecentDirective, String>>>

Purpose: This unit test ensures the getDirectideById logic is working correctly. This tests flow transformation, DAO lookup, and correct directive text.

Test Case Categories:

- *Valid inputs:* Recent directive entries referencing existing directive IDs.
- *Boundary cases:* Empty recent directive list returned from database.
- *Edge cases:* Directive ID exists in history, but no longer exists in the directive table.

Sample Tests:

- Mapping a valid directive ID correctly returns its directive text alongside the history record.
- Mapping an empty directive list correctly returns an empty StateFlow list.
- Mapping a directive ID that no longer exists returns "Unknown Directive" safely.

Unit Under Test: days: StateFlow<List<ActiveDay>>

Purpose: This test is made to make sure that the database for active hours is correctly exposed. This confirms the DAO flow is collected correctly, the values propagated to the ViewModel are correct, and that StateFlow emits the expected list.

Test Case Categories:

- *Valid inputs:* Database returns a list containing multiple ActiveDay records.
- *Boundary cases:* Database returns an empty ActiveDay list.
- *Edge cases:* Database returns entries with inactive status for all days.

Sample Tests:

- Retrieving active days correctly emits the database list through StateFlow.
- Retrieving an empty database list returns an empty StateFlow safely.
- Retrieving all inactive days correctly reflects inactive status in emitted values.

Unit Under Test: updateDay(day: ActiveDay)

Purpose: This verifies that the active hours DAO actually executes. This checks that the coroutine launches correctly, the Dao insert is called, and that the correct object is passed.

Test Case Categories:

- *Valid inputs:* Updating active hours for a valid weekday entry;
- *Boundary cases:* Updating startHour at 0; updating endHour at 23.
- *Edge cases:* Updating a day entry that already exists in the database; Updating a day entry with identical values to the stored record.

Sample Tests:

- Updating an ActiveDay entry correctly calls the DAO insert function.
- Updating an ActiveDay entry with a midnight start time stores correctly.
- Updating an ActiveDay entry with unchanged values does not crash the database update process.

Integration Testing

For KIMinder360, integration testing serves as the bridge between isolated unit testing of individual components and the live final version, with cohesion between features. While unit tests confirm that our scheduling algorithms and data models work in a vacuum, integration testing ensures that these components communicate properly across the MVVM layers and with Android System Services. As we move forward with adding functionality to the KIMinder360 app that requires more cross component integration, our focus is moving beyond verifying "if it works" and shifting to "how does it work together". Specifically, this consists of looking for defects in lifecycle management, background task persistence, and data flow between the Room database and Jetpack Compose UI.

Our approach focuses on the "critical path" of the user experience with KIMinder360, the moment a directive is scheduled, when it triggers as a notification, and when the user records a response. Failures at these moments, such as a notification failing to fire because of a configuration mismatch in WorkManager or a failure to record data to the database, would fundamentally break KIMinder360's core functionality and nullify our value proposition.

Key Integration Points

To ensure system stability, we have identified the following primary points for verification:

1. **ViewModel – DAOs – Room Database:** Validates that UI actions (like completing a directive) are correctly funneled through the DAOs and persisted in the Room DB, and that the UI reactively updates via Kotlin Flows.
2. **WorkManager – Application Logic – NotificationManager:** This is the most important pathway. It ensures that the background worker can correctly access user preferences (DataStore)/ database (Room), select a valid directive from the database, and trigger a system notification at the intended time.
3. **Database – Scheduling Logic:** Verifies that changes to "Active Hours" or "Categories" in the user settings are immediately picked up by the scheduling logic to prevent notifications during non-active hours.
4. **UI (Jetpack Compose) – ViewModel:** Ensures that the state produced by the ViewModel correctly drives the UI components, particularly when handling complex user controls or displaying directive history data.

Test Environment and Setup

The integration testing environment is designed to be comprehensive and repeatable, utilizing the AndroidX Test library suite.

1. **Execution:** Tests are executed on Android Emulators with an API of 30 or above and physical devices to account for manufacturer specific battery optimization behaviors that might affect WorkManager.
2. **Database Setup:** We use Room In-Memory Databases. This allows us to spin up a fresh database instance for every test suite, ensuring that data from one test does not leak into another.
3. **Dependency Injection:** We utilize Hilt (Dagger) for dependency injection, allowing us to swap out real Notification Managers with "Test Fakes" when we need to verify that a notification would have been sent without actually cluttering the device's notification shade.
4. **Data Cleanup:** Using the `@Before` and `@After` JUnit annotations, we can reset DataStore preferences and clear all scheduled WorkManager tasks before each test scenario to maintain a clean slate state.

Integration Scenarios

The following scenarios tie our technical components to real world user flows and acceptance criteria.

Scenario 1: Update Active Hours and Reschedule Directives

- **Integration Point:** UI – Database – WorkManager
- **Feature:** Modify scheduling parameters via user customization.
- **Scenario Description:** A user changes their "Active Hours" to start later in the morning.
- **Integration Steps:**
 1. The user updates the "Start Time" in the Compose based active hours screen.
 2. The ViewModel writes the new start time to the database.
 3. A listener detects the change.
 4. The existing worker in the daisychain is canceled and rescheduled to an appropriate time according to the new time window.
- **Expected Results:**
 - The database persists the new time correctly.
 - The Directive Scheduler log shows the next scheduled notification which falls within the new time range.
 - No notifications are triggered during the previously active (now inactive) window.
- **Failure Handling:**
 - Persistence errors in the database trigger a "Settings could not be saved" Toast.
 - If WorkManager fails to reschedule, it falls back to a known safe action.

Scenario 2: Background Directive Trigger and Local Persistence

- **Integration Point:** WorkManager – Room DB – NotificationManager
- **Feature:** Directives are sent as notifications reliably.
- **Scenario Description:** A background task triggers, selects a directive, and notifies the user.
- **Integration Steps:**
 1. WorkManager wakes up at the scheduled interval.
 2. The Worker queries the Room Database for an available directive based on the user's selected categories.
 3. The Worker passes the directive text to the NotificationManager.
 4. The Worker logs the "Sent" status back into the Room database.
 5. The Worker schedules a new time to wake up.
- **Expected Results:**

- The system notification is successfully posted to the Android tray.
- The next time for the worker to wake up and send another directive is scheduled.
- **Failure Handling:**
 - If the database is empty, the Worker gracefully exits and retries later.
 - If notification permissions are revoked, the Worker logs the failure and ceases scheduling.

Scenario 3: User Response and Engagement Tracking

- **Integration Point:** Notification UI – ViewModel – Room DB
- **Feature:** Ability to respond to directives.
- **Scenario Description:** A user taps on a directive notification and responds in the app.
- **Integration Steps:**
 1. The user interacts with the directive from the notification or the directive history log.
 2. The User fills out the completion checkbox and/or the text box before clicking submit.
 3. The UI calls a function in the ViewModel to update the database
 4. The viewmodel updates the room database entry for that recent directive with the user's response.
 5. The response data is loaded into the UI when the user accesses that directive again.
- **Expected Results:**
 - The database correctly stores the response data.
 - The UI reflects the change immediately without requiring a manual refresh.
- **Failure Handling:**
 - Database write conflicts are handled via an "Upsert" strategy to prevent data loss.
 - If the app is killed during the write, the Room transaction ensures data integrity is maintained.

Usability Testing

For KIMinder 360, usability testing evaluates how users interact with the system to match their preferences, interpret directives, and respond to notifications. While unit testing and integration testing mostly verify internal functionalities and component interactions, usability testing focuses on clarity, efficiency, and accessibility of the features that users encounter. The primary goals are to identify interactions that are confusing, make sure that users can successfully complete tasks given by the app, and improve the overall user experience of the application so that situational awareness training is natural and easy to follow.

KIMinder360 is designed for mobile users who have varying levels of technological experience but who have a general understanding of basic smartphone functionalities and features, such as notifications and interface settings. The application introduces a new interaction style where users receive directives throughout the day, so they need to understand how scheduling works and respond within short time periods. Since these interactions are quick and often appear when users are busy with other tasks, the interface needs to be clear and simple to use. Otherwise, users may miss directives, configure their settings incorrectly, and/or lose interest in the application. For this reason, usability testing focuses on important user tasks and interactions to make sure that the system is easy to use.

Usability testing is done before acceptance so that issues with user experience can be caught early on. That way, we can fix anything that is difficult to follow before the final evaluation. By the time of acceptance, the application should be clear and easy to follow for general mobile users.

Usability Testing Methods

Users: Students/general mobile users (primary), project client/mentor (secondary)

Goals:

- Check if users can understand and respond to directives without confusion.
- Identify where interactions become unclear with notifications, scheduling, and settings.
- Improve how smooth and straightforward the main workflows feel.

Method:

- Task-based usability sessions where users complete common actions in the app.
- Simulated notifications are used to test responses in real-time.
- Little guidance is given to see how natural the app feels.

- Short sessions (10-15 minutes) that focuses on the core features.
- Receive client feedback and internal reviews.

Sessions:

- 3-5 student participants (primary users)
- 1 client feedback session
- 1 expert review (mentor or non-UI team member)

Tasks:

- Set up preferences and notification settings
- Respond to a directive from a notification
- View recent and past directives within history
- Update settings after interacting with a directive

Measures:

- How easily users move between tasks without getting stuck.
- How quickly users react to and complete directive interactions.
- Points where users hesitate or take the wrong action.
- Overall impressions of the user experience (whether it's clear vs confusing).

Follow-up:

- Identify and review patterns seen across sessions rather than single issues.
- Shift focus first to problems with directive interaction and notifications.
- Apply changes before the next testing round.
- Track issues that are constant to confirm they are resolved over time.

Testing Workflow and Quality Controls

This section outlines the rigorous standards and systematic processes KIMinder360 uses to ensure that defects are identified, tracked, and resolved before reaching the user. Since our app relies on background execution and precise timing, we cannot afford to wait for bugs to appear. We need a structured policy for enforcing our code quality.

Bug Reporting Requirements

Every bug report must include the following:

1. **Unique ID and Title:** A concise summary and unique identifier for the issue
2. **Associated User Story:** What feature does the issue affect?
3. **Environment Details:** Android API level, device or emulator type, and the version of the app being used.
4. **Reproduction Steps:** What actions were taken to get to the bug output?
5. **Expected and Actual Result:** What should've happened versus what actually happened/what went wrong.
6. **Severity Level:** How much does it impact the core functionality of the app?

Severity Levels

Level	Severity	Description	Action
S1	Critical	App crashes, data loss in DB or Datastore, failure of the scheduling system.	Must be fixed before any further development can continue.
S2	Major	Core functionality works with major issues. Notifications fire at the wrong times or user settings customization is broken.	Must be fixed before the next development cycle/demo.
S3	Minor	UI glitches, minor performance stutters, or theme styling issues.	Fix if time permits during this development cycle. If not, then it can be delayed until the next sprint.
S4	Inconsequential	Grammatical or spelling mistakes as well as cosmetic inconsistencies.	Low priority, it can wait until we have time for polishing.

Development Cycle Controls

To maintain a clear understanding of when our development cycles can be started or ended, we have clear criteria for each.

Entry Criteria (Starting a Cycle)

Before beginning a new cycle:

1. **Code Review Completion:** All new code must be reviewed for architectural consistency with our implementation of MVVM
2. **Unit Test Baseline:** 100% of existing unit tests must pass
3. **Hilt Modules Updated:** Dependency injection modules must be configured for any new services

Exit Criteria (Ending a Cycle)

A cycle is deemed complete when:

1. Zero S1 or S2 Defects: No critical or major bugs that affect core functionality are present.
2. Integration Pass: All integration scenarios are passed on at least one physical device.
3. Documentation Update: Any changes to the room database schema or scheduling logic are clearly documented.

Conclusion

In conclusion, this software testing plan establishes an organized approach to validating the functionality, reliability, and usability of the KIMinder360 application. By combining unit testing, integration testing, and usability testing, the plan ensures that both the internal logic of the system and the user experience are thoroughly evaluated. Unit tests target critical business logic such as directive selection, scheduling behavior, and data validation, while integration tests verify that these components operate cohesively across the MVVM architecture and Android system services. Usability testing further complements these efforts by ensuring that real users can interact with the system intuitively and efficiently in real-world scenarios.

Additionally, the inclusion of clearly defined testing environments, repeatable test setups, and realistic integration scenarios ensures that the system is validated under conditions that closely mirror actual usage. The emphasis on critical system pathways—particularly directive scheduling, notification delivery, and response tracking—directly addresses the highest-risk areas of the application. Combined with usability testing, this strategy helps ensure that the application not only functions correctly but also remains accessible and engaging for its target users.

Finally, the defined quality controls, including strict bug reporting standards, severity classifications, and development cycle entry and exit criteria, provide a strong framework for maintaining code quality throughout the development lifecycle. By enforcing that all critical defects are resolved and that core features are validated before progression, the team minimizes the risk of unstable releases. Overall, this testing strategy will allow the team to deliver a highly reliable, functional, and user-friendly product in KIMinder360 that effectively meets its goal of improving users' situational awareness skills in everyday environments.